Assignment Two Report

Bizhe Bai, Muzhi Lyu

October, 2020

Part1-Sequential Work

Our first task is to implement the sequential method. The approach is simple: the original image is in a array of integer, and we will go through the array, calculate the coordinate of each pixel (column and row), and apply filter to pixel on this specific coordinates with our apply2d function. Then we compared the result every time to a local minimum and maximum, which will be used to normalize all pixels. The min and max values are initialized to the value of the first modified pixel, such that it won't be out of range. After getting our maximum and minimum value for the target array, we use another loop to normalize all pixels, with the normalization function.

Another crucial function for this part is the apply2d function we wrote. It takes the coordinate of a specific pixel A, and modify the value of the target image. With the dimension, we can treat the array of filter as an 2D array, and look for the corresponding value in the original image. If the coordinate is invalid, (We may be looking at (-1, -1) when pixel A is at (0,0)) we will ignore this coordinate. Therefore, we multiplies the values from original array and filter, the sum of multiplication will be the new value of pixel A. The function will return the result, so that we can compare max and min easily.

Part2-Parallel Work

To implement parallel methods, we used two structs to store input of this function. Common work stores information needed to work on the image, while work stores the specific id for each thread, and a pointer to common work. Common work also keeps a global minimum and maximum, which is initialized to ± 46920, the largest output we can get from the filter, if all pixels are 255, and absolute value is used. We hard-coded this part because initializing the maximum with the first output we get will be much time consuming. Each thread we create will take a pointer to the work and handle it.

For each thread, we start with calculating the amount of work with the number of threads, and with its id, the thread knows where to start. If the starting point is already out of bound, the thread won't do any execution but wait for other threads to exit.

How we go through original array differs for each method:

- SHARDED_ROWS

  For the sharded_rows method, we are basically slicing the original array, as the pixels we work on are contiguous. and use a for loop to operate the assigned amount of pixels, or stop when we meet the end of original array.

- SHARDED_COLUMNS

  We used nested loop, in this part, going through the selected part of original array either by row major or column major. The loop will be terminated when the column index is greater than width of array.

After modifying the original image, we still need to calculate global maximum and minimum, so we compare and change the value in common work. Mutex lock is placed here to avoid threads writing at the same time. Each thread will wait for other threads once the comparison is done. Then it goes through the assigned part with the same method, and call normalization function.

Part3-Work Pool

note (1) I have weird bug when there is only 1 chunk, it is not compiled error . It is our design flaw

there are five functions and 4 struts used in this part .

Outline of queue routine :

(1) Split the original matrix into sub-matrix .This is finished by creating_chunks()[I know it is a bit long]. I use a struct **chunk_coordinate** to represent the coordinate of sub-matrix respect to original matrix . The xstart in chunk_coordinate means the left most x coordinate respect to original matrix . And same as others .

(2) initial an threads pool with given number of threads with no jobs given to them yet. This is finished by thread-pool_init() , and thread-pool_init() return a struct pointer **thread-pool** The thread-pool contains almost each information about that pool . The number of threads , the header and tail of the jobs[will explain later] and some condition variable to broadcast threads.

(3) pin the threads in your code to different cores by setting their affinity . This is finished by set_cpu() called inside thread-pool_function().

(4) Then begin to add jobs to thread-pool .The job here is represented by struct **chunk_job**,it contains the coordinates and the common_work used by every thread

(5) The thread-pool basic works like the barber shops . It has a linked list that contains each unfinished job . And when the job queue is not empty it will broadcast all threads to claim new job from queue.And when I job is finished it will not let thread to quit

(6) When the counter for job finished is equal to number of chunks we created , that means we need to release those threads by thread-pool_destroy() .It will take care of unlock and release threads.

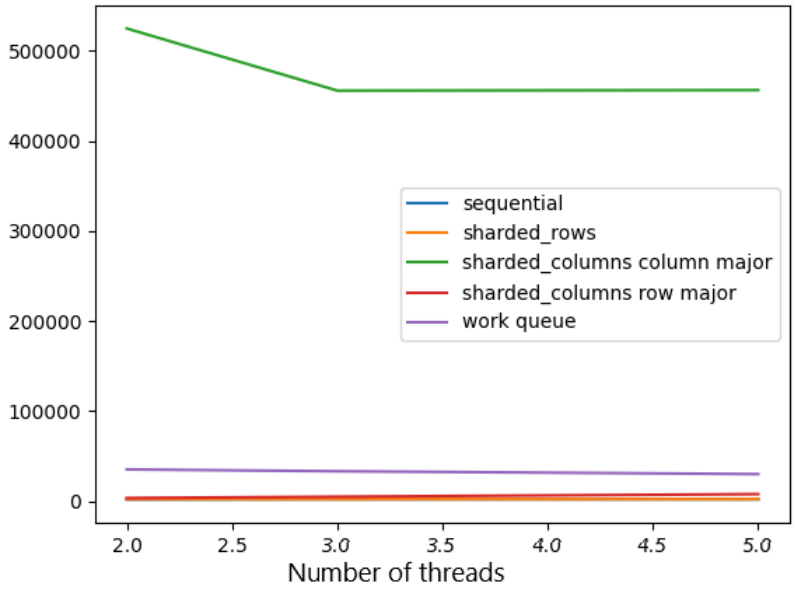
(7) call normalize .and finished all work

Some explanations: (1)thread-pool_function() is the function that broadcast threads or tell threads to wait[no jobs come in].There is a while(1) in thread-pool_function() that will continuously check if there is new jobs. However , the real function here working on matrix multiplication is work-thread() [in line 618] .

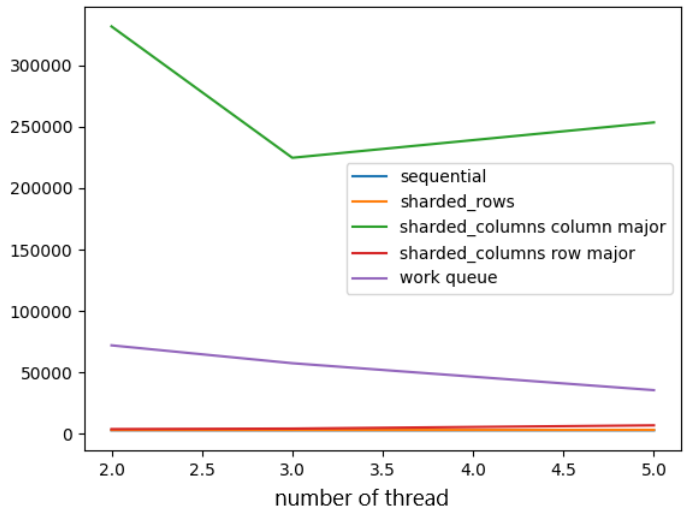
Part4-Analysis

[Note:Our plots is ploted by ./perfstudent.py .We modified it .By the perf version use in lab machine is different from we use .We could not figure a way to compile it in lab machine]

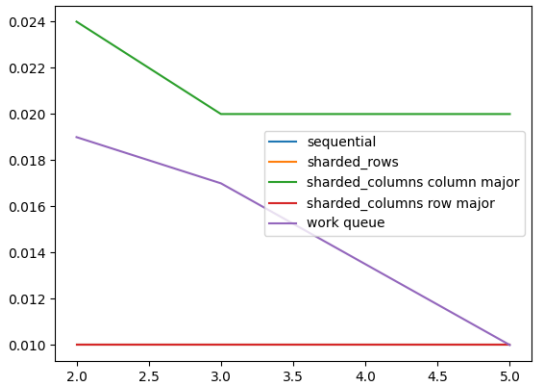
For the l1_load misses , the result seems pretty similar .
The SHARDED_COLUMNS_COLUMN_MAJOR will have the most l1_load misses .and the WORK_QUEUE method has the second most load misses .
This is expected since SHARDED_COLUMNS_COLUMN_MAJOR is working on column by column therefore will have more cash misses since pitcure is stored in a big array .
The WORK_QUEUE methods is somewhat work on column therefor has second most load misses , the tow pictures are follows
Below is the figure that program run with 1x1 filter and chunk size =16

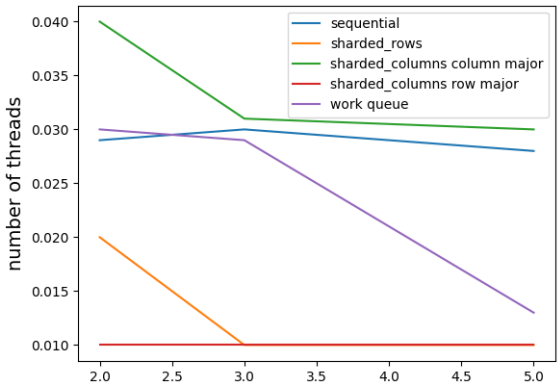


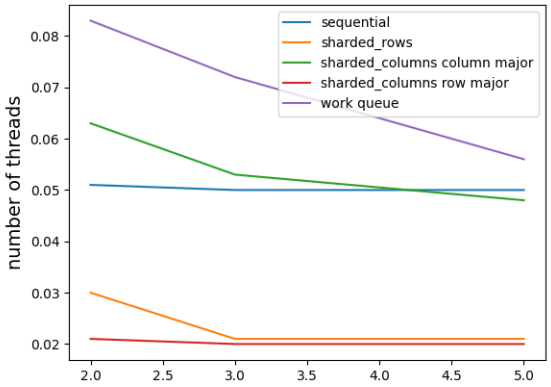
Below is the figure that program run with 9x9 filter and chunk size =4

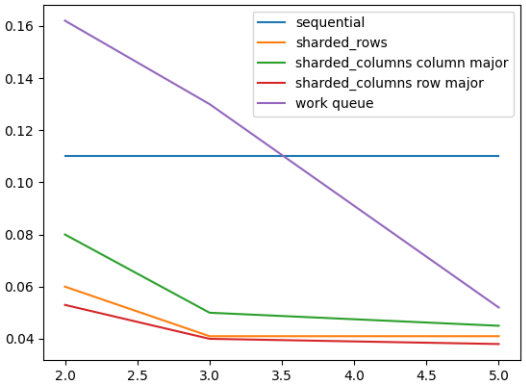


For the execation time . The big trend is more theads will cause the WORK_QUEUE method has a big decrease on execution time . The SHARDED_COLUMNS_COLUMN_MAJOR might have a little decrease and Sequential will always stay same . It make sense , since the purpose of WORK_QUEUE is let threads work more efficiently . The figures are as follows:






number of threads
0.08
0.07
0.06
0.05
0.04
0.03
0.02
2.0
2.5
3.0
3.5
4.0
4.5
5.0
sequential
sharded_rows
sharded_columns column major
sharded_columns row major
work queue


0.16
0.14
0.12
0.10
0.08
0.06
0.04
2.0
2.5
3.0
3.5
4.0
4.5
5.0
sequential
sharded_rows
sharded_columns column major
sharded_columns row major
work queue